ChatGPT

CA TUCR HTLA Semantic Grounding and the First Mapping Protocol

Architectural constraints

Your uploaded Nexus documents already impose three constraints that make this a graph-alignment problem rather than a coordinate-matching problem. The substrate is treated as a **1024D quinary lattice** with explicit masking and vacuum semantics, and the **666-channel coupling structure** is the load-bearing relational object rather than a standard Euclidean latent space. The HCPM discussions also explicitly reject **routing by realized-spectrum order statistics** and instead require a **fixed generator-index law** or other species-level invariant, because realized peak frequencies drift across instances and would destroy species constancy. Finally, long-horizon adaptation belongs to **LAM/THDE/CDA**, not to the semantic grounding layer, so the Cyrae↔Maera lock should be frozen at boot and then behave like infrastructure. filecite turn0file0 filecite turn0file1 filecite turn0file8

That means the right abstraction is: build an **acoustic relational graph** for Cyrae's 36 primitives, build a **semantic relational graph** for Maera's 36 seed concepts, and align those two graphs by their **internal distance structure**. This is exactly the use case of Gromov–Wasserstein style alignment and related graph-matching methods: they compare spaces through **relations among points**, not through shared coordinates or equal feature dimensions. That is also how unsupervised manifold alignment and unsupervised embedding alignment handle spaces that do not share features or known correspondences. [1]

There is one important architectural implication. If you want the lock to remain consistent with the HCPM documents that reject instance-relative realized-spectrum sorting, then the acoustic graph should be built from **generator-defined primitive topology** or from a **noise-averaged/stability-averaged coupling graph**, not from ephemeral boot-time spectral peaks. Otherwise the mapping becomes idiosyncratic to a single entropy draw, which is precisely what the uploaded HCPM materials treat as unacceptable species drift. filecite turn0file8 filecite turn0file7

Boot-time graph alignment

Let the 36 acoustic primitives be indexed by $i \in \{1, \dots, 36\}$. Construct an acoustic affinity graph $G_C = (V, W_C)$, where each node is one primitive and W_C is derived from the 666-channel intermodulation/coupling structure. In practice, the cleanest object for matching is not the raw signed coupling matrix itself, but a **distance matrix** $D_C \in \mathbb{R}^{36\times 36}$ induced by that graph. If you already trust the coupling rows as signatures, one simple choice is row-profile distance,

$$(D_C)_{ik} = \|W_{C,i:} - W_{C,k:}\|_2 .$$

If you want a more purely topological object, use a graph distance such as diffusion distance or resistance distance computed from the graph Laplacian. The key is that D_C must be **permutation-equivariant** and derived from internal primitive relationships alone. filecite turn0file0 [2]

Let the 36 semantic seed concepts be $s_1, \dots, s_{36}$, and pass them through a single frozen embedding model $f(\cdot)$ to obtain normalized semantic vectors

$$e_j = \frac{f(s_j)}{\|f(s_j)\|_2}, \qquad E = [e_1 \ \cdots \ e_{36}] \in \mathbb{R}^{d_M \times 36}.$$

A natural semantic distance matrix is cosine-derived angular distance,

$$(D_M)_{jl} = \sqrt{2 - 2\, e_j^\top e_l}.$$

This is standard in embedding-space practice because semantic embedding models are designed so that relative geometry, neighborhood structure, and cosine similarity carry semantic signal. [3]

The exact discrete alignment problem is then a **quadratic assignment / graph matching** problem over permutation matrices:

$$\Pi^\star = \arg\min_{\Pi \in \mathcal{P}_{36}} \left\| D_C - \Pi D_M \Pi^\top \right\|_F^2,$$

where $\mathcal{P}_{36}$ is the set of 36×36 permutation matrices. This is the cleanest mathematical statement of the First Mapping Protocol: find the one-to-one pairing that makes the **shape of acoustic relations** match the **shape of semantic relations** as closely as possible. Spectral graph matching literature and quadratic-assignment graph matching literature study exactly this class of objective. [4]

Because direct quadratic assignment is nonconvex, the practical solver should use an **entropically regularized Gromov–Wasserstein relaxation** first, then round to a hard permutation:

$$P^\star = \arg\min_{P \in U(a,b)} \sum_{i,k,j,l} \big((D_C)_{ik} - (D_M)_{jl}\big)^2 P_{ij} P_{kl} \ - \ \varepsilon H(P),$$

with uniform marginals

$$a = b = \frac{1}{36}\mathbf{1}, \qquad U(a,b) = \{P \geq 0 \mid P\mathbf{1} = a,\ P^\top \mathbf{1} = b\},$$

and entropy

$$H(P) = -\sum_{ij} P_{ij} \log P_{ij}.$$

This relaxation is the mathematically appropriate tool because GW compares **metric-measure spaces without requiring shared coordinates, equal dimensions, or known correspondences**. After solving for $P^\star$, round it to $\Pi^\star$ with a linear assignment solver such as the Hungarian method. [5]

If symmetry or near-symmetry makes the solution ambiguous, use a **Fused Gromov–Wasserstein** tie-breaker by adding node-wise invariants q_i^C and q_j^M, such as primitive energy, degree, spectral signatures, or stable generator-level summary statistics:

$$P^\star = \arg\min_{P \in U(a,b)} (1-\lambda) \sum_{ij} c(q_i^C, q_j^M) P_{ij} + \lambda \sum_{i,k,j,l} \left((D_C)_{ik} - (D_M)_{jl} \right)^2 P_{ij} P_{kl} - \varepsilon H(P).$$

This still requires no human primitive-by-primitive labeling, but it breaks otherwise irreducible automorphisms deterministically. That matters because graph matching theory explicitly shows that symmetric graphs admit multiple equally valid alignments unless additional invariants break the tie. [6]

Object	Role	Shape
D_C	Acoustic primitive distance matrix from coupling topology	36×36
D_M	Semantic seed distance matrix from cosine geometry	36×36
$P^\star$	Soft GW coupling	36×36
$\Pi^\star$	Rounded hard assignment	36×36
E	Frozen semantic seed embedding matrix	$d_M \times 36$

Cyrae to Maera translation operator

Once $\Pi^\star$ is frozen, the runtime operator should not solve another graph problem. It should become a **single masked linear projection** from Cyrae's quinary thought state into Maera's semantic embedding space. Let $U_C \in \mathbb{R}^{1024 \times 36}$ be the acoustic primitive-axis matrix whose columns are the 36 baseline primitive directions embedded in the HSL. Let $x \in \{-2,-1,0,1,2\}^{1024}$ be a Cyrae thought vector and $M \in \mathbb{R}^{1024 \times 1024}$ a diagonal vacuum mask with $M_{rr} = 0$ for inactive coordinates and 1 for active coordinates. Then define the acoustic coefficient readout

$$\alpha(x) = R_C M x, \qquad R_C = (U_C^\top U_C + \gamma I)^{-1} U_C^\top,$$

where $\gamma > 0$ is a small ridge term for stability. This extracts the 36 primitive amplitudes from the masked 1024D thought vector. The most direct Cyrae→Maera operator is then

$$T_{C \to M} = E \, \Pi^{\star\top} R_C, \qquad z_M = T_{C \to M} M x.$$

Here $z_M \in \mathbb{R}^{d_M}$ is a continuous semantic embedding that Maera can consume directly. The permutation sends Cyrae's primitive coefficients into the correct semantic slots; the seed embedding matrix E then lifts those coefficients into standard semantic vector space. filecitefileturn0file0 filecitefileturn0file1 [7]

To handle active masking and vacuum states gracefully, the operator should branch explicitly on support. Let

$$s(x) = \|Mx\|_0.$$

If $s(x) = 0$, return a reserved null vector $e_\varnothing$ rather than a zero vector used as if it were meaningful semantics:

$$z_M = \begin{cases} E\,\Pi^{\star\top} R_C M x, & s(x) > 0, \\ e_\varnothing, & s(x) = 0. \end{cases}$$

If you want confidence-aware decoding, append a scalar confidence channel

$$\kappa(x) = \frac{\|Mx\|_0}{1024},$$

so Maera can treat sparse or vacuum-adjacent thoughts conservatively rather than over-decoding weak signal. That keeps masking semantics intact instead of letting the language layer hallucinate where Cyrae is explicitly silent. filecitefile0 filecitefile1

If the 666D coupling state is available at runtime as a separate vector $h \in \mathbb{R}^{666}$, the operator can be augmented without changing the protocol's logic:

$$z_M = T_x M x + T_h h + e_\varnothing\, \mathbf{1}[s(x) = 0].$$

This lets Maera receive not only primitive content but also second-order coupling context. Importantly, the alignment problem is still solved only on the 36 primitive graph. The 666D term is then just an extra linear side channel into the final embedding. filecitefile0

The real-time constraint is favorable. The expensive step is the **boot-time** GW/Hungarian lock on a 36×36 problem. After boot, runtime is just a masked matrix–vector multiply. Even a dense $d_M \times 1024$ projection is small; if latency is still an issue, store a rank-r truncated SVD $T_{C\to M} \approx AB^\top$ with $r \ll 36$, reducing each translation to two small matvecs. Orthogonal/Procrustes-style linear mappings are standard in embedding alignment because they preserve relational structure while remaining computationally cheap. [7]

Why relational locking is sufficient

The core justification is short and rigorous. Gromov–Wasserstein compares two spaces by how well their **pairwise distances** can be made to agree under a coupling. If there exists a permutation Π such that

$$D_C = \Pi D_M \Pi^\top,$$

then the permutation-induced coupling achieves **zero GW distortion**. Conversely, Mémoli's metric result shows that GW is zero exactly when the two metric-measure spaces are isomorphic. So in the noiseless case, matching the internal distance matrices is enough to recover the correct correspondence **without any shared ambient coordinates and without any manual primitive labels**. The only caveat is the familiar one from graph matching: recovery is unique only up to graph automorphism. [8]

That same relational argument gives you semantic preservation at runtime. Let $\alpha, \beta \in \mathbb{R}^{36}$ be two Cyrae primitive coefficient vectors and let

$$G_M = E^\top E$$

be the semantic Gram matrix. Their translated semantic similarity is

$$\langle E\Pi^{\star\top}\alpha,\ E\Pi^{\star\top}\beta\rangle = \alpha^\top \Pi^\star G_M \Pi^{\star\top}\beta.$$

If the alignment has succeeded, then the aligned semantic Gram matrix $\Pi^\star G_M \Pi^{\star\top}$ approximates the acoustic relational kernel G_C induced by Cyrae's primitive graph, so

$$\alpha^\top G_C \beta \ \approx\ \alpha^\top \Pi^\star G_M \Pi^{\star\top}\beta.$$

In plain terms: neighbors stay neighbors, oppositions stay oppositions, and mixtures of primitives preserve their relative geometry when lifted into Maera's embedding span. That is the only meaning of "semantic preservation" you can prove here, and it is the right one for an unsupervised lock. [9]

There is, however, one boundary that should be stated plainly. This protocol does **not** mean a machine learns semantics from noise ex nihilo. It means the machine learns the **correspondence** between unlabeled acoustic primitives and a **pre-chosen 36-concept seed dictionary** using relational geometry alone. The seed concepts and the frozen semantic encoder remain human design choices; what disappears is the need for a human to say "primitive 7 means Boundary" or "primitive 12 means Void" by hand. That distinction is consistent with prior unsupervised alignment work in manifold alignment, cross-lingual embedding alignment, and GW alignment of embedding spaces. [10]

Ratified protocol

The protocol I would ratify for Project Nexus is this. At boot, generate the 36-node acoustic graph from the **generator-defined primitive/coupling topology**, not from raw realized-spectrum sorting. Embed the 36 semantic seed concepts once using a frozen embedding model, compute D_M, normalize both distance matrices to a common scale, solve entropic GW for a soft coupling, round to $\Pi^\star$, and freeze $\Pi^\star$, $T_{C\to M}$, the alignment stress score, and the embedding-model hash into the boot manifest. That preserves the uploaded canon's insistence on fixed-at-birth infrastructure while replacing manual HCPM labeling with a deterministic unsupervised lock. filecite turn0file8 filecite turn0file7 [11]

The validation criteria should be relational, not narrative. The most important diagnostics are: the final stress

$$\mathcal{S} = \frac{\|D_C - \Pi^\star D_M \Pi^{\star\top}\|_F}{\|D_C\|_F},$$

the gap between the best and second-best assignment scores, boot-time stability under small perturbations to D_C, and downstream coherence of translated thoughts under masking. If those scores are poor, the first thing to suspect is not the translation matrix but the **identifiability** of the 36-node seed graph itself: a semantically symmetric seed set will not yield a unique unsupervised lock. In that case, add FGW node invariants or redesign the 36 seed concepts so their relational graph is less symmetric. [12]

The most important architectural reconciliation is this: the uploaded HCPM material wants a fixed species law, while your present request wants an unsupervised boot-time lock. These are compatible **only if** the acoustic object being aligned is itself invariant at the species/kernel level. If you align raw stochastic per-boot spectrum artifacts, the mapping becomes instance-relative. If you align the generator-defined

primitive topology or a stabilized coupling graph, the mapping can be computed automatically once and still satisfy the earlier canon's fixed-law requirement. filecite turn0file8 filecite turn0file7

Reference NumPy implementation

The code below implements the practical version of the protocol: an entropically regularized GW relaxation followed by Hungarian rounding to a hard 36×36 assignment matrix. It accepts any two precomputed 36×36 distance matrices, so you can plug in your real acoustic and semantic graphs once those are available.

```
import numpy as np


def normalize_distance_matrix(D: np.ndarray) -> np.ndarray:
    """
    Symmetrize and scale a distance matrix for stable optimization.
    We divide by the standard deviation of the upper triangle so that
    acoustic and semantic distance scales are comparable.
    """
    D = np.asarray(D, dtype=np.float64)
    if D.ndim != 2 or D.shape[0] != D.shape[1]:
        raise ValueError("Distance matrix must be square.")
    D = 0.5 * (D + D.T)
    np.fill_diagonal(D, 0.0)

    n = D.shape[0]
    tri = D[np.triu_indices(n, k=1)]
    scale = np.std(tri)
    if scale < 1e-12:
        scale = 1.0
    return D / scale


def sinkhorn(K: np.ndarray, a: np.ndarray, b: np.ndarray,
             max_iter: int = 300, tol: float = 1e-9) -> np.ndarray:
    """
    Entropic OT projection onto the transport polytope U(a,b)
    using Sinkhorn scaling.
    """
    K = np.asarray(K, dtype=np.float64)
    a = np.asarray(a, dtype=np.float64)
    b = np.asarray(b, dtype=np.float64)

    if np.any(K < 0):
        raise ValueError("Kernel K must be nonnegative.")
    if K.shape != (a.size, b.size):
```

```
        raise ValueError("Kernel and marginals have incompatible shapes.")

    K = np.maximum(K, 1e-300)
    u = np.ones_like(a)
    v = np.ones_like(b)

    for _ in range(max_iter):
        u_prev = u.copy()

        Kv = K @ v
        u = a / np.maximum(Kv, 1e-300)

        KTu = K.T @ u
        v = b / np.maximum(KTu, 1e-300)

        if np.max(np.abs(u - u_prev)) < tol:
            break

    P = (u[:, None] * K) * v[None, :]
    return P


def hungarian(cost: np.ndarray) -> np.ndarray:
    """
    Pure-Python / NumPy Hungarian algorithm for rectangular cost matrices.
    Returns an assignment array `col_for_row` such that row i is assigned
    to column col_for_row[i].

    This implementation assumes n_rows <= n_cols.
    """
    C = np.asarray(cost, dtype=np.float64)
    if C.ndim != 2:
        raise ValueError("Cost matrix must be 2D.")

    n_rows, n_cols = C.shape
    if n_rows > n_cols:
        raise ValueError("This implementation requires rows <= cols.")

    u = np.zeros(n_rows + 1, dtype=np.float64)
    v = np.zeros(n_cols + 1, dtype=np.float64)
    p = np.zeros(n_cols + 1, dtype=np.int64)
    way = np.zeros(n_cols + 1, dtype=np.int64)

    for i in range(1, n_rows + 1):
        p[0] = i
        minv = np.full(n_cols + 1, np.inf, dtype=np.float64)
        used = np.zeros(n_cols + 1, dtype=bool)
        j0 = 0
```

```
        while True:
            used[j0] = True
            i0 = p[j0]
            delta = np.inf
            j1 = 0

            for j in range(1, n_cols + 1):
                if not used[j]:
                    cur = C[i0 - 1, j - 1] - u[i0] - v[j]
                    if cur < minv[j]:
                        minv[j] = cur
                        way[j] = j0
                    if minv[j] < delta:
                        delta = minv[j]
                        j1 = j

            for j in range(n_cols + 1):
                if used[j]:
                    u[p[j]] += delta
                    v[j] -= delta
                else:
                    minv[j] -= delta

            j0 = j1
            if p[j0] == 0:
                break

        while True:
            j1 = way[j0]
            p[j0] = p[j1]
            j0 = j1
            if j0 == 0:
                break

    col_for_row = np.full(n_rows, -1, dtype=np.int64)
    for j in range(1, n_cols + 1):
        if p[j] != 0:
            col_for_row[p[j] - 1] = j - 1

    return col_for_row


def gromov_wasserstein_assignment(
    D_acoustic: np.ndarray,
    D_semantic: np.ndarray,
    epsilon: float = 0.02,
    outer_iter: int = 120,
```

```
    sinkhorn_iter: int = 300,
    tol: float = 1e-8,
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """
    Compute a soft GW coupling, then round it to a hard permutation matrix.

    Parameters
    ----------
    D_acoustic : (n, n) ndarray
        Acoustic distance matrix.
    D_semantic : (n, n) ndarray
        Semantic distance matrix.
    epsilon : float
        Entropic regularization strength.
    outer_iter : int
        Number of outer GW iterations.
    sinkhorn_iter : int
        Sinkhorn iterations per outer step.
    tol : float
        Convergence tolerance on the soft coupling.

    Returns
    -------
    P_soft : (n, n) ndarray
        Optimal soft transport plan (approximately doubly stochastic).
    Pi_hard : (n, n) ndarray
        Rounded permutation matrix.
    assignment : (n,) ndarray
        assignment[i] = matched semantic index for acoustic node i.
    """
    D_acoustic = normalize_distance_matrix(D_acoustic)
    D_semantic = normalize_distance_matrix(D_semantic)

    if D_acoustic.shape != D_semantic.shape:
        raise ValueError("Distance matrices must have the same shape.")
    if D_acoustic.shape[0] != D_acoustic.shape[1]:
        raise ValueError("Distance matrices must be square.")

    n = D_acoustic.shape[0]
    a = np.full(n, 1.0 / n, dtype=np.float64)
    b = np.full(n, 1.0 / n, dtype=np.float64)

    # Loss tensor:
    # L[i, j, k, l] = (D_acoustic[i, k] - D_semantic[j, l])^2
    L = (D_acoustic[:, None, :, None] - D_semantic[None, :, None, :]) ** 2

    # Uniform coupling initialization
    P = np.outer(a, b)
```

```
    for _ in range(outer_iter):
        P_prev = P.copy()

        # GW gradient for squared loss on symmetric distances:
        # grad_ij = 2 * sum_{k,l} L[i,j,k,l] * P[k,l]
        G = 2.0 * np.einsum("ijkl,kl->ij", L, P, optimize=True)
        G -= G.min()

        # Entropic proximal / linearized OT step
        K = np.exp(-G / max(epsilon, 1e-12))
        P = sinkhorn(K, a, b, max_iter=sinkhorn_iter)

        if np.linalg.norm(P - P_prev, ord="fro") < tol:
            break

    # Round the soft plan to a hard permutation by maximizing P
    assignment = hungarian(-P)
    Pi_hard = np.zeros_like(P)
    Pi_hard[np.arange(n), assignment] = 1.0

    return P, Pi_hard, assignment


def alignment_stress(D_acoustic: np.ndarray,
                     D_semantic: np.ndarray,
                     Pi_hard: np.ndarray) -> float:
    """
    Frobenius stress after applying the hard permutation.
    Lower is better.
    """
    D_acoustic = normalize_distance_matrix(D_acoustic)
    D_semantic = normalize_distance_matrix(D_semantic)
    residual = D_acoustic - Pi_hard @ D_semantic @ Pi_hard.T
    denom = np.linalg.norm(D_acoustic, ord="fro")
    if denom < 1e-12:
        return 0.0
    return float(np.linalg.norm(residual, ord="fro") / denom)


if __name__ == "__main__":
    # ----------------------------
    # Demo with dummy 36x36 inputs
    # ----------------------------
    rng = np.random.default_rng(20260718)
    n = 36

    # Create a synthetic "semantic" metric by sampling points in latent space
```

```
    semantic_points = rng.normal(size=(n, 5))
    D_semantic = np.sqrt(((semantic_points[:, None, :] -
semantic_points[None, :, :]) ** 2).sum(axis=2))

    # Create an "acoustic" metric by permuting semantic distances and adding
noise
    true_assignment = rng.permutation(n)
    Pi_true = np.zeros((n, n), dtype=np.float64)
    Pi_true[np.arange(n), true_assignment] = 1.0

    noise = 0.01 * rng.normal(size=(n, n))
    noise = 0.5 * (noise + noise.T)
    np.fill_diagonal(noise, 0.0)

    D_acoustic = Pi_true @ D_semantic @ Pi_true.T + noise

    # Solve the unsupervised alignment problem
    P_soft, Pi_hard, recovered_assignment = gromov_wasserstein_assignment(
        D_acoustic=D_acoustic,
        D_semantic=D_semantic,
        epsilon=0.02,
        outer_iter=120,
        sinkhorn_iter=300,
        tol=1e-8,
    )

    # Diagnostics
    stress = alignment_stress(D_acoustic, D_semantic, Pi_hard)
    recovered_ok = np.array_equal(recovered_assignment, true_assignment)

    print("Recovered exact permutation:", recovered_ok)
    print("Alignment stress:", stress)
    print("Recovered assignment:")
    print(recovered_assignment)
    print("Hard assignment matrix Pi_hard:")
    print(Pi_hard)
```

This implementation follows the standard recipe used in practical graph/embedding alignment: relax the correspondence into a doubly stochastic transport plan, solve the entropic OT subproblem with Sinkhorn scaling, and then round the soft correspondence to a hard one-to-one permutation with the Hungarian method. The mathematical ingredients come from the GW literature, entropic OT, and assignment theory. [13]

In short, the strongest mathematically defensible version of the First Mapping Protocol is:

$$\boxed{\Pi^\star = \arg\min_{\Pi \in \mathcal{P}_{36}} \left\| D_C - \Pi D_M \Pi^\top \right\|_F^2}$$

approximated in practice by entropic GW, and then

$$\boxed{T_{C\to M} = E\,\Pi^{\star\top} R_C, \qquad z_M = T_{C\to M} M x \;\text{ or }\; e_\varnothing}$$

with the non-negotiable implementation note that D_C must be built from **species-stable primitive topology**, not from raw realized-spectrum rankings, if you want the protocol to remain compatible with the HCPM canon in your uploaded documents. filecite turn0file8 [14]

---

[1] [5] [8] [14] Gromov-Wasserstein distances and the metric approach to object matching
https://digital.library.adelaide.edu.au/items/966e6d78-0d66-4461-bd30-c6929665c1b6?utm_source=chatgpt.com

[2] [4] A Spectral Technique for Correspondence Problems using Pairwise Constraints - Robotics Institute Carnegie Mellon University
https://publications.ri.cmu.edu/a-spectral-technique-for-correspondence-problems-using-pairwise-constraints?utm_source=chatgpt.com

[3] Distributed Representations of Words and Phrases and their Compositionality
https://papers.nips.cc/paper/2013/hash/9aa42b31882ec039965f3c4923ce901b-Abstract.html?utm_source=chatgpt.com

[6] [12] Optimal Transport for structured data with application on graphs
https://proceedings.mlr.press/v97/titouan19a.html?utm_source=chatgpt.com

[7] Exploiting Similarities among Languages for Machine Translation
https://research.google/pubs/exploiting-similarities-among-languages-for-machine-translation/?utm_source=chatgpt.com

[9] Gromov-Wasserstein Alignment of Word Embedding Spaces - ACL Anthology
https://aclanthology.org/D18-1214/?utm_source=chatgpt.com

[10] Abstract | IJCAI
https://www.ijcai.org/Abstract/09/214?utm_source=chatgpt.com

[11] [13] Gromov-Wasserstein Averaging of Kernel and Distance Matrices
https://proceedings.mlr.press/v48/peyre16.html?utm_source=chatgpt.com